

The Noticing Gap

Every company between roughly ten and two hundred people has the same hole in it: work that quietly stops moving because *nobody's job is to notice*. This is a system that closes it — and a description of exactly which clients it is worth selling to.

What it is an attention layer over the tools a client already has
Status built, tested, running daily on live company data

THE PROBLEM

Capacity is not what is failing

Ask anyone in a fifty-person company whether they are busy and you will learn nothing. Everyone is busy. What actually fails is narrower, and almost none of it arrives in an inbox looking like a thing to do:

- **A supplier said "I'll follow up" and never gave a date.** Six weeks later nobody has thought about it since. Undated promises are the ones that vanish — precisely because there is nothing to be late against.
- **A decision only one person can make is blocking four people's work**, and none of them have said so, because each is only aware of their own piece.
- **Something got finished and nothing was ever closed**, so it keeps occupying a slot in everyone's head and a line on every list.
- **Two numbers moved in a way that matters** — spend up, traffic flat — but they live in two different systems and nobody had both open on the same day.

- **A five-minute approval that would have freed a colleague's entire day** got done at 4:30pm.

Every one of these is a small failure of *noticing*, and each one costs far more than the minute it would have taken to catch. In a large organisation, noticing is somebody's job — a chief of staff, an ops team, a programme manager. Below that scale it is nobody's.

The gap opens after the founder stops knowing everything, and closes when someone is finally paid full-time to know it.

That band — call it ten to two hundred people — is the entire addressable market for this, and it is very large.

WHY THE CURRENT STACK MISSES IT

Three categories of tool, none of which is aimed at this

The first objection you will meet is "we already have software for that." The client almost certainly does have all three of these, and none of them is built to close this gap. Being precise about why is the whole pitch.

CHAT ASSISTANTS

Answer when asked

They know everything about the world and nothing about this business — no memory of what was promised last month, no state of its own. They are excellent, and entirely reactive.

You cannot ask about the thing you forgot.

TRACKERS & CRMS

Assume someone maintains them

Honest systems of record that are only as true as the last person who updated them. At this size, nobody does so reliably — and a tracker that is sixty per cent current is worse than none, because it is confidently wrong.

The truth is in threads, not in the tracker.

Optimise throughput

They move things between systems and measure themselves in actions fired. If attention is the binding constraint, pushing more volume through the same funnel makes the constraint tighter, not looser.

More tasks is the wrong direction.

What is missing is a fourth thing: something that **reads what already happened** across mail, chat, calls and the numbers; **keeps its own state** rather than trusting anyone to maintain one; and is measured not on what it produces but on what it takes off people's plates.

A system that produces more tasks has made things worse.

That one line is the design constraint the whole build follows from, and it is the sentence to lead with in a first meeting. It rules out almost everything else in the category, and clients recognise it immediately — because they have all bought something that failed this test.

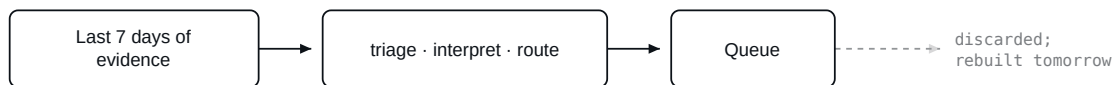
WHAT IT DOES

Read the week, then remember it

Every morning it reads the last several days of email, chat and call transcripts, works out which of it implies real work, and decides who should own each piece — not by asking, but from what the company's own history shows about who actually does what.

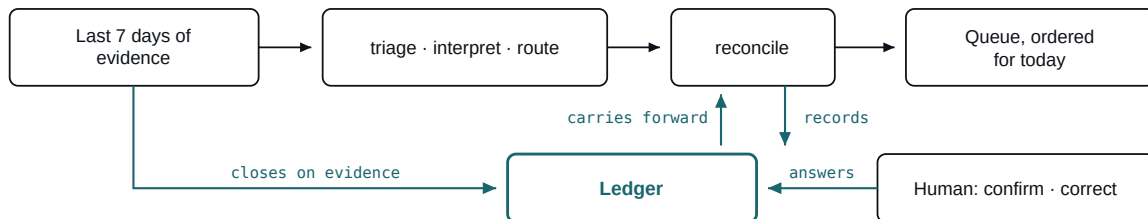
Then it does the part nearly every build skips. Producing a ranked list from a week of evidence is the easy half. The problem shows up in month two: a system with no memory rebuilds the same list every morning from the same evidence. Nothing can ever be finished, and nothing can be compared to yesterday.

WHAT MOST BUILDS SHIP



Yesterday's finished work returns on today's evidence. The queue only grows, and the client stops opening it.

WHAT SURVIVES MONTH TWO



Unfinished work survives its window and is re-scored against today's clock — which is how an approaching deadline finally lifts it.

The same intake pipeline, with and without a ledger. The teal edges are the entire difference: evidence can *close* work, unfinished work is *carried forward* rather than rebuilt, and a person's answers are *recorded* so the same question is never asked twice. Nearly every prototype in this category ships the top diagram.

What the client sees is a single page, opened once in the morning: **what to do and in what order today**, budgeted against the hours actually left; **everything outstanding**, including what has gone quiet; **the handful of numbers that would change a decision**; and **a short list of moves the business could choose to take on**, each one grounded in something that actually happened.

It runs on the connectors the client already has — mail, chat, calls, accounting, storefront, marketing — so there is no new system for anyone to adopt and no data migration to negotiate.

WHAT IT IS NOT

not a chatbot

not a CRM

not a BI dashboard

not an agent that acts on its own

not a headcount replacement

HOW IT DECIDES

Five rules that carry most of the weight

These are the parts worth walking a client through, because each one corresponds to a way the obvious approach gets it wrong.

01

Ownership is six roles, not one

Real work has a primary owner, a project manager, a decision maker, an approver, collaborators, and very often an external counterparty. Collapsing that into a single assignee field is where most tools lose the plot.

The consequence that matters commercially: **needing the founder is not the same as being the founder's job**. A supply problem may need one person's decision and another's execution. Assign it to the decision maker and you have quietly moved a week of operations onto the wrong desk — which is exactly the outcome the client is paying to avoid.

02

Attention is priced by what it costs, not what is at stake

Approving a fifty-thousand-dollar production run and approving a social post both take about five minutes. Ordering someone's day by value at stake therefore gets it systematically wrong.

Each item is priced by its **mode** — decide, approve, delegate, do, follow up — and the plan is budgeted against the hours genuinely left in the day. A plan that needs nine hours at 3pm is not a plan, and pretending otherwise is how a daily tool stops being opened in week three.

03

The value of unblocking someone decays through the day

Approving a colleague's work at 9am buys a full day of their output. The identical approval at 4:30pm buys nothing until tomorrow.

So a five-minute approval that frees someone else outranks an hour of the principal's own higher-scoring work in the morning — and stops outranking it by late afternoon. **The order changes with the clock**, which no static priority list can do. This is usually the moment a sceptical client leans forward.

A number is only true once its period is closed

Most businesses close their books monthly, on a lag. Until a month closes its expense side is incomplete, so any profit figure drawn from it is fiction wearing the costume of fact — and it is the figure every naive dashboard shows first.

Profit and expense conclusions come only from closed periods, compared against the previous closed period, with the period named so that "now" is never assumed. Order-level data is unaffected and stays separate, because the two bases legitimately differ and blending them produces a number that is true on neither.

A change must clear the noise before it is reported

A weekly conversion rate moves several tenths of a point on nothing at all. A system that announces every wobble teaches its reader to ignore it — and is then ignored on the day it matters.

Movements are converted into **standard errors**, and anything below three sigma is reported as what it is: within chance. Restraint is the feature. A dashboard that never says *this is probably nothing* is not measuring, it is decorating.

THE TRUST SURFACE

Three refusals

All of the above is judgement, and judgement can be wrong. What makes this safe to leave running — and what answers the objection every client raises within ninety seconds — is not its accuracy. It is what it declines to do when it is confident.

NEVER

Sends as anyone

Not one message leaves in a person's name. When a supplier is overdue, the output is a draft addressed to whoever owns the chasing, quoting what the supplier actually said. The system contributes the noticing — the part that fails — never the relationship.

NEVER

Closes what it cannot safely close

Work wrongly left open is visible and costs a moment to dismiss. Work wrongly closed disappears, along with whatever it was worth. Inference closes only what is both confident and cheap to be wrong about; anything consequential becomes a one-click question with the evidence quoted.

NEVER

Treats silence as completion

Work that stops appearing has usually either been finished with nobody saying so, or quietly died. Those need opposite responses. After three weeks of silence an item leaves the daily plan and is asked about *once* — a daily "is this still live?" is the same nagging in slower form.

THE DIFFERENTIATOR

What a from-scratch build gets wrong

This is the section that closes the sale, and the one to be most honest in. A competent developer can build the first seventy per cent of this in a few weeks. The last thirty per cent is what decides whether the client is still opening it in month three — and it is made of specific mistakes you can only find by running against real correspondence.

Three that were caught in live deployment, each of which now has a rule and a regression test behind it:

1 · The number that was the wrong sign

On its first real run the system led with a confident headline: the company was losing money and ad spend had exploded. It was reading the month in progress, whose bills were half-entered. The closed books said close to the opposite.

WHAT THE OPEN MONTH SHOWED

-\$50K

Net, with ad spend apparently up 61%. Drawn from a month whose bills were not yet entered.

WHAT THE CLOSED BOOKS SAID

+\$35K

Profit *improving*, with spend up 23%. Nearly the opposite conclusion, from the same report.

Rounded from a live deployment. The fix was not a better model — it was a rule about when a number is allowed to be believed. Every accounting connector on the market will hand you the first figure by default.

2 · The status update that closed a whole project

Completion has to be inferred from evidence, because at this size nobody reliably closes anything by hand. Run against real chat traffic, the obvious implementation read this and marked a four-channel rollout finished:

TEAM CHAT · OPERATIONS CHANNEL

Images were updated across the marketplaces. Retailer A — waiting on approval, do you want me to list the new SKU? Marketplace B — can we activate it in draft? **Marketplace C — done.** Marketplace D — waiting for image processing.

One line out of four was finished, and that line closed everything. The rule that came out of it: **any unfinished marker anywhere in a message settles it** — that is a progress report, not a completion, however confident the rest of it sounds. A second rule fell out of the same investigation: **a message cannot finish the work it asked for**, because overlapping read windows mean Monday's request gets read again on Tuesday, and it was closing the very task it had opened.

3 · The promise the system was blind to

The follow-up engine had been written months earlier and had never once fired. It measured lateness against a due date — and the most common promise in real business

correspondence has no date at all:

EMAIL · EXTERNAL PARTNER

I'll look into those and follow up.

Nothing to be late against, so it was never chased. Which means the promises being dropped were exactly the ones the system was structurally incapable of seeing. The fix requires memory: the clock starts when the promise was first heard. That single change surfaced a real commitment at **ten business days outstanding**, quoting the partner's own words back to whoever has to send the nudge.

A client cannot buy this scar tissue. They can only pay for it once, or discover it themselves over six months of a system slowly losing their trust.

QUALIFICATION

Who this is worth selling to

"Any small business" is both true and useless. There is a sharper predictor, and it is neither industry nor headcount:

How many outside parties can make this client a promise?

Each one is a place a ball gets dropped, and none of them are in the client's systems. A company with three suppliers and one sales channel does not need this. A company running a contract manufacturer, a logistics provider, two retail buyers, a media agency, a marketplace and a lab has a dropped-ball surface no individual can hold in their head — and no software they own is watching it, because the evidence lives in threads.

Count the outside parties who owed them something last month.

Over eight, and nobody's job is tracking them? **Strong fit.** Under four? Send them a shared spreadsheet and keep your reputation.

If their two busiest people took a week off, how much would stop?

Most of it? **Attention is the binding constraint**, and everything here is aimed squarely at it.

Where would a new hire look to find out what is outstanding?

"Read my email and ask around" means **they have evidence but no state** — the exact gap this fills.

Who signs off, and how often are they the reason something waited?

A named bottleneck who is genuinely willing to open one page each morning. **Without that person, do not sell it.**

CLIENT SHAPE	FIT	WHAT CARRIES THE VALUE
Consumer brand or manufacturer	STRONG	Contract manufacturers, logistics partners, retail buyers, agencies and marketplaces all making commitments by email, a founder as the decision bottleneck, and real numbers scattered across three systems that nobody reconciles.
Agency or professional services firm	STRONG	Commitments <i>are</i> the product, and "who owns this client thread" is genuinely ambiguous rather than merely undocumented. The follow-up engine is worth more here than anywhere else — and it is the easiest value to demonstrate in week one.
B2B with long implementation cycles	STRONG	Multi-party work where a dropped ball is expensive and stays invisible for weeks. Deadline-aware re-ranking matters most when the gap between cause and consequence is long.
Import, brokerage, distribution, franchise ops	STRONG	Enormous counterparty surface, nearly everything by email, and approval boundaries that genuinely differ per transaction rather than existing as policy theatre.
Construction, events, field services	GOOD	Same counterparty density and the same thread-bound truth, but expect more of the evidence to arrive by phone and photo. Scope the connector work honestly before quoting.

CLIENT SHAPE	FIT	WHAT CARRIES THE VALUE
Regulated small operation (clinic, lab, food)	PARTIAL	Approval routing and the never-send rule become the headline rather than a footnote — but the finance and funnel layers are largely irrelevant, so roughly half the system sits idle. Price accordingly.
Work already in a system of record people update	POOR	If their tracker is genuinely accurate, they do not need inference over evidence — they need a better view of what they already have. This entire architecture exists because the truth lives in threads.
High-volume repetitive operations	POOR	Support queues, claims, ticketed fulfilment. The economics are per-ticket, so throughput automation is the right tool. Allocating attention optimises a variable that is not binding.
Two hundred plus with a real ops function	POOR	A learned capability graph adds nothing over an accurate directory, and somebody is already paid to do the noticing. The gap this fills is specifically the gap where nobody's job is to notice.
Founder-only or two-person shop	POOR	No delegation surface, so ownership routing, approval classes and the unblocking rule are all inert. What remains is a to-do list with unusually good manners.

DELIVERY

What the engagement actually looks like

The shape is consistent across clients because the expensive part is never the code. It is learning how a specific company really works — which is also the part the client values most, and the part they will try to skip.

Map

~1 week

Interviews and reading, almost no code. Who is actually good at what, as against what their title says. Which outside parties exist and who owns each relationship. What needs approval and by whom. Where the real decisions get made. **Clients get value from this phase even if they stop here** — most have never had it written down, and saying so up front makes the engagement easy to start.

Connect

days

Wire up the connectors they already have, and decide which numbers earn a place. The discipline here is subtraction: the test for every figure is **would seeing this change what someone does in the next hour**. Most candidates fail it, and cutting them is what keeps the page worth opening.

Calibrate

2–4 weeks

Run it daily against real traffic and correct it where it is wrong. Every correction teaches it something specific. **This is the phase clients want to skip and cannot** — it is where the wrong owner, the false completion and the over-eager signal all surface, and it is the reason a bought system beats a built one.

Hand over

ongoing

It runs. The configuration is theirs to edit in plain files — people, capabilities, counterparties, approval thresholds — so the model of their business stays current without a change request. Natural retainer scope: new connectors, new signals, and re-mapping as the company grows.

The split between what transfers and what gets rebuilt is unusually clean, because everything client-specific was deliberately pushed into configuration rather than code. This is what makes the second engagement much cheaper to deliver than the first.

Transfers unchanged

- The ledger: identity across runs, carry-forward, dormancy
- Completion detection and its refusals
- Follow-up cadence, escalation, the undated-promise clock
- Deduplication across mail, chat and calls
- Approval classes and the never-send boundary
- The daily plan, including the decay rule
- Statistical gating on every reported movement

Rebuilt per client

- The capability graph — who is actually good at what
- Business areas and their typical decision makers
- Which counterparties exist and who owns each relationship
- Connectors, and which numbers earn a place
- Finance rules: when the books close, what "true" means in them
- Priority weights and what triggers an escalation

OBJECTION HANDLING

The three you will hear every time

"I am not letting AI email my suppliers."

Correct, and neither is this. It never sends anything in anyone's name — that is architectural, not a setting someone can flip. When a supplier is overdue it prepares a draft for the person who owns that relationship, quoting what the supplier said. They send it, or they do not.

This objection is the client telling you what they are afraid of. Answering it properly usually converts them, because nothing else they have been pitched has a good answer.

"We already have a project tool for this."

Good — keep it. A tracker is where work goes once somebody has decided it exists and who owns it. This is about the deciding, which happens in email and chat and calls, and it can write into whatever they already use.

Then ask the qualifying question: *how current is it, honestly?* Nobody at this size answers "completely", and the answer is the sale.

"Could our developer just build this?"

The first seventy per cent, yes, in a few weeks. Do not pretend otherwise — the claim will not survive contact with their CTO, and conceding it buys the credibility for what follows.

The last thirty per cent is the difference between a system they still open in month three and one they quietly stop opening. It is not clever architecture; it is a specific list of mistakes — closing a project on a status update, chasing a promise that was already kept, announcing a conversion wobble that was statistical noise — that you only find by running against real correspondence for weeks. They can pay for that once, or discover it themselves.

WHERE IT STANDS

Built, not proposed

297 TESTS, ALL PASSING	57 MODULES THAT TRANSFER UNCHANGED
10 CONFIG FILES REWRITTEN PER CLIENT	0 MESSAGES EVER SENT ON ANYONE'S BEHALF

This is a working system running daily on live company data across mail, chat, call transcripts, accounting, storefront and marketing — not a prototype and not a deck. Every rule described above exists because the system got the corresponding thing wrong against real data first, and the fix is in the code with a test holding it there.

The right first conversation is not a demo. It is the four qualifying questions above, asked in a client's own words, and a walk through the moment a five-minute approval at 9am is worth more than an hour of the founder's own work. If they lean forward at that, they have the gap. If they do not, they have somebody whose job it is to notice — and they do not need this.

Figures shown are rounded and drawn from a live deployment; they illustrate the mechanism rather than serving as a financial statement. No client names, correspondence or identifying detail appear in this document.